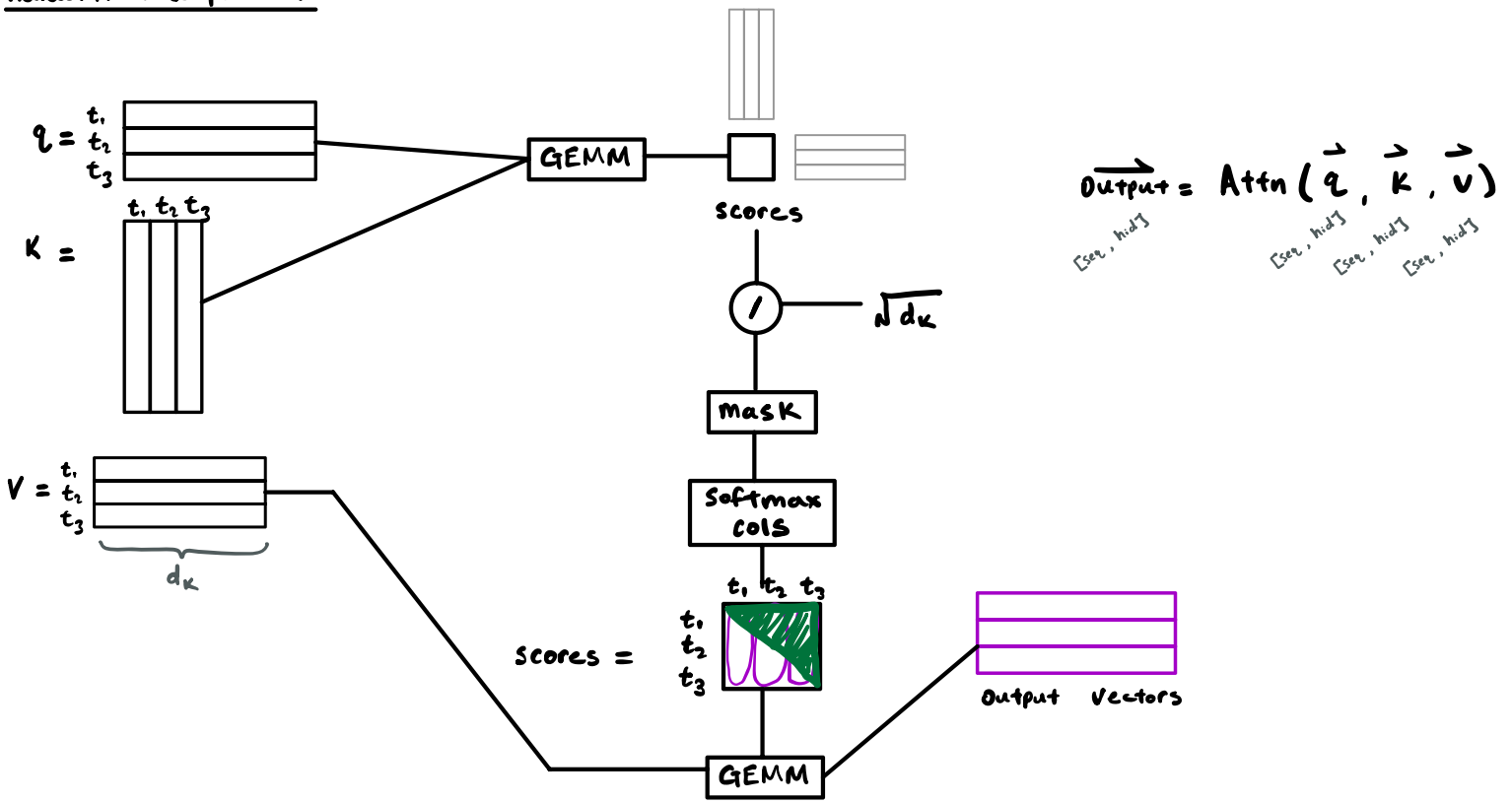
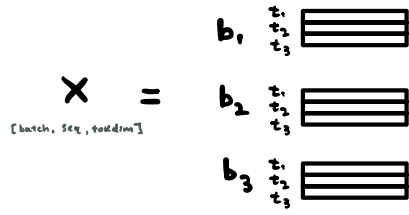


Review: Attn Computation

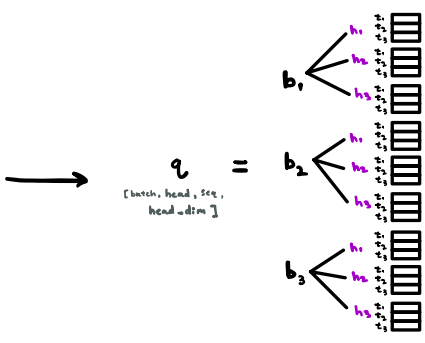
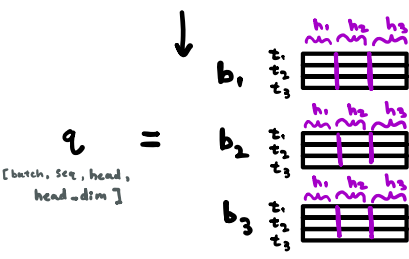
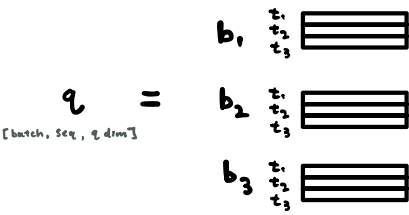


MHA

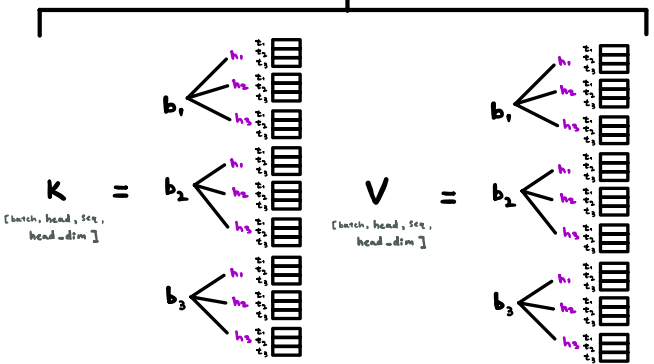


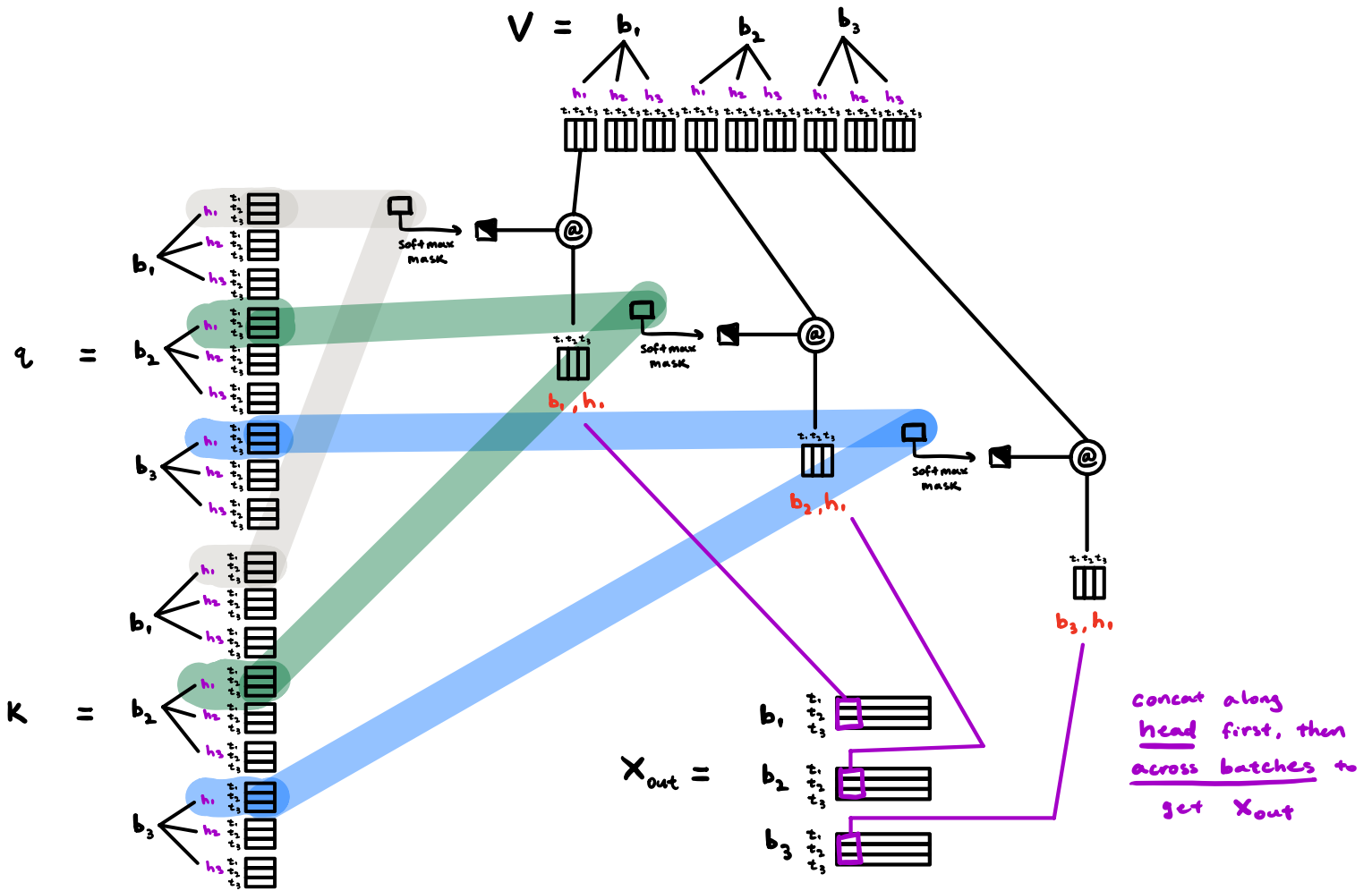
First, reshape to reveal the new HEAD dimension!

↓ Q proj.

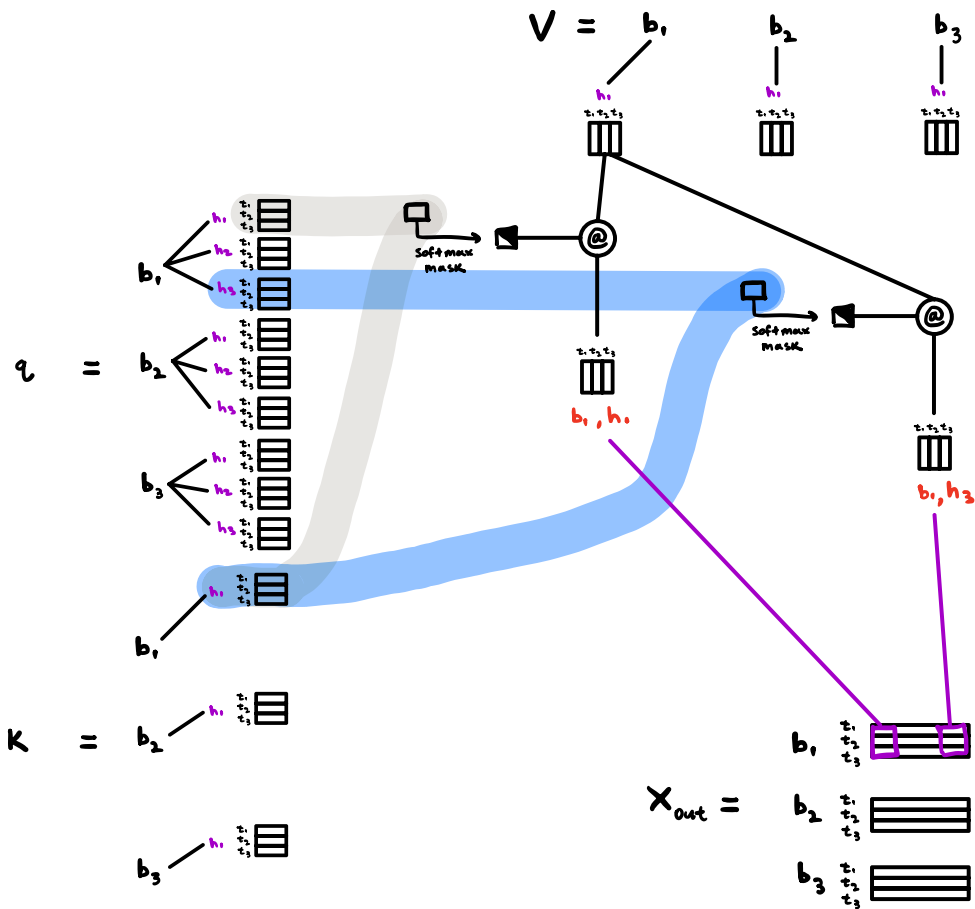


Produce K, V in similar fashion





GQA - same idea but # KV heads < # Q heads



FlashAttention

The diagrams above demonstrate attention is really just 2 matmuls:

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Naive implementation is SLOW:

- materialize full QK^T
- softmax each row
- gemm result with V

~ 3 separate kernels!

+ we materialize full QK^T which doesn't fit in HBM

Trick: online softmax

'Safe' softmax is

$$\begin{bmatrix} e^{x_0 - x_{\max}} / \sum_{i=0}^n e^{x_i - x_{\max}} \\ e^{x_1 - x_{\max}} / \sum_{i=0}^n e^{x_i - x_{\max}} \\ e^{x_2 - x_{\max}} / \sum_{i=0}^n e^{x_i - x_{\max}} \\ \vdots \\ e^{x_n - x_{\max}} / \sum_{i=0}^n e^{x_i - x_{\max}} \end{bmatrix}$$

In code, we need 2 reads ("Scans" / full iterations) over memory

```
x_max = -inf
for i in range(len(x)):
    x_max = max(x_max, x[i])
```

} find max element

```
exp_sum = 0
for i in range(len(x)):
    vec_value = exp(x[i] - x_max)
    exp_sum += vec_value
    ans[i] = vec_value
```

} compute exp. terms

Online Softmax lets us combine the 2 Scans using a 'running max':

```
x_max = -inf
exp_sum = 0
```

```
for i in range(len(x)):
```

```
    new_max = max(x_max, x[i])
```

```
    exp_sum = exp_sum * exp(x_max - new_max) + exp(x[i] - new_max)
```

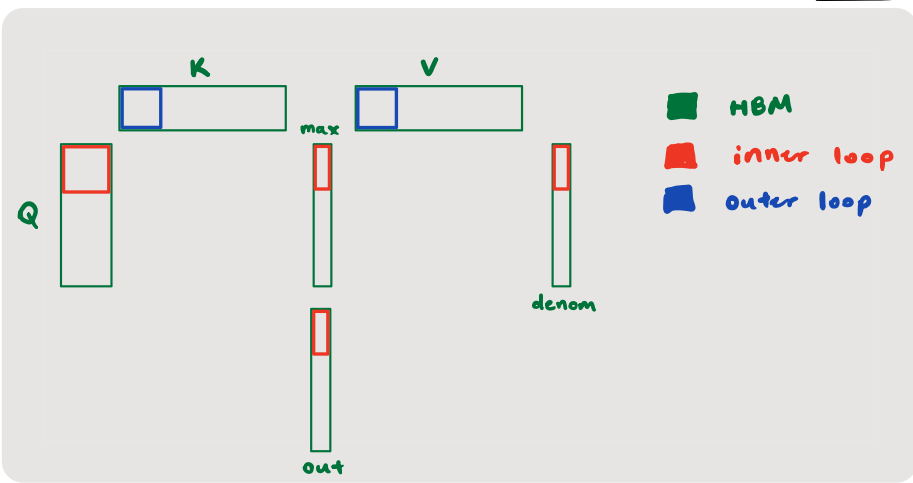
```
    x_max = new_max
```

notice

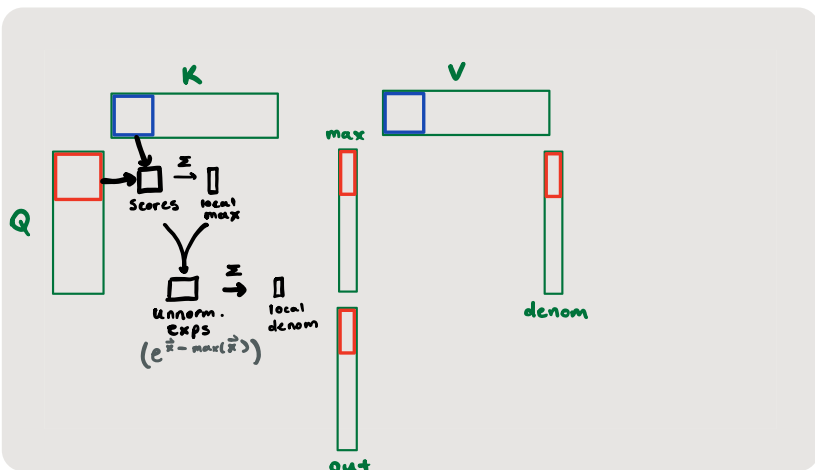
$$\left(\sum_i e^{\vec{x}[i] - \text{old_max}}\right) \cdot \left(e^{\text{old_max} - \text{new_max}}\right) = \sum_i e^{\vec{x}[i] - \text{new_max}}$$

"old" max
"running" max

This 'correction factor' trick lets us tile attention!



```
for K_tile, V_tile in split-by-cols(K, V):
    for Q_tile, Out_tile, max_tile, denom_tile in split-by-rows(...):
```

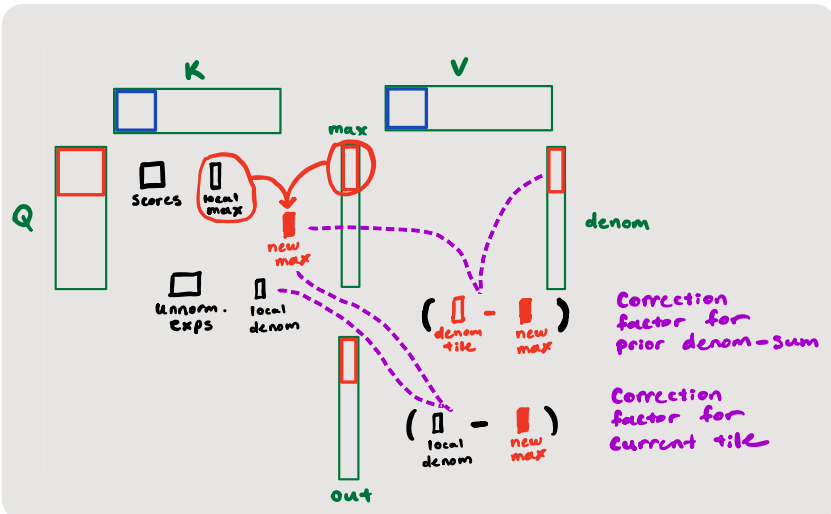


$$\text{Scores}_{\text{tile}} = (Q_t @ K_t^T) \times \text{scale}$$

$$\text{local max} = \text{rowmax}(\text{Scores}_{\text{tile}})$$

$$\text{Unnorm-exps} = \exp(\text{Scores}_{\text{tile}} - \text{local max})$$

$$\text{local denom} = \text{rowsum}(\text{Unnorm-exps})$$



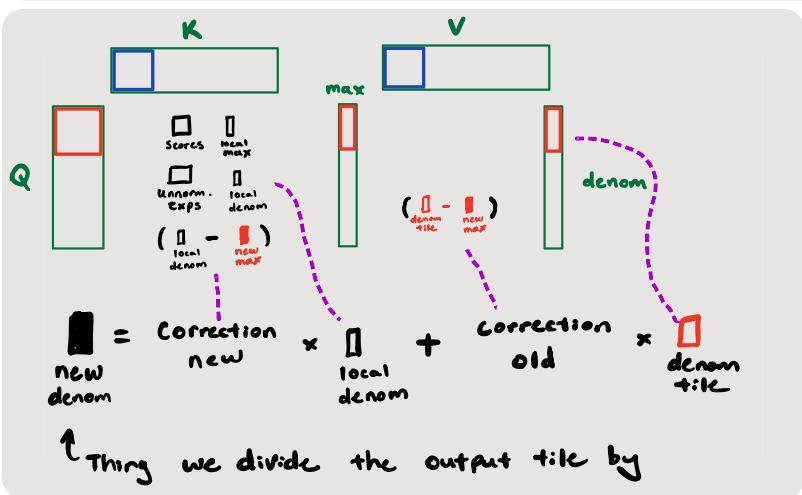
old denoms using max_tile

new tile uses local max

$$\text{new-max} = \max(\text{max}_{\text{tile}}, \text{local max})$$

$$\text{Scale-old} = \exp(\text{max}_{\text{tile}} - \text{new-max})$$

$$\text{Scale-new} = \exp(\text{local max} - \text{new-max})$$

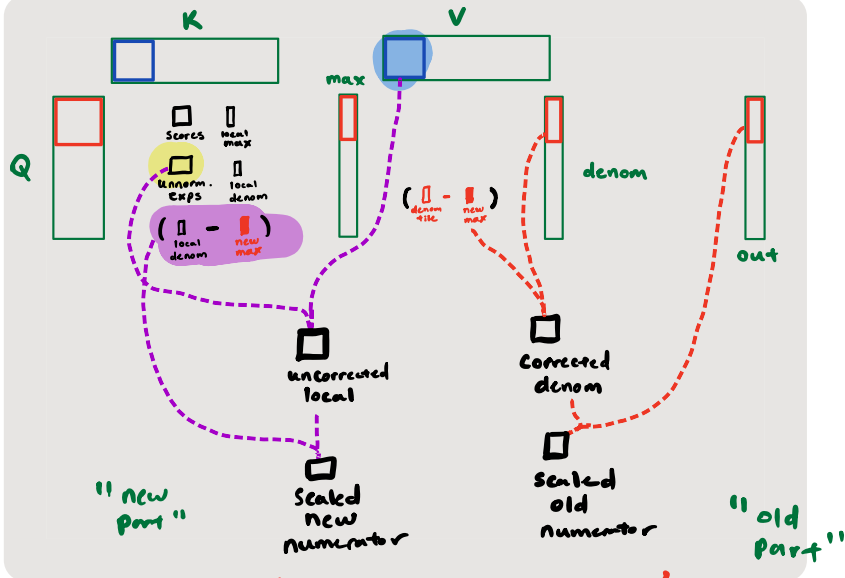


$$\text{new denom} = \left(\text{Scale}_{\text{old}} \times \text{denom}_{\text{tile}} \right) + \left(\text{Scale}_{\text{new}} \times \text{local denom} \right)$$

Reminder:

$$\text{Numerator Expected} = e^{s_1 - M} \vec{v}_1 + e^{s_2 - M} \vec{v}_2 + \dots$$

$$\text{Denom Expected} = e^{s_1 - M} + e^{s_2 - M} + \dots$$



using these SRAM-loaded inputs, you can compute your desired output tile

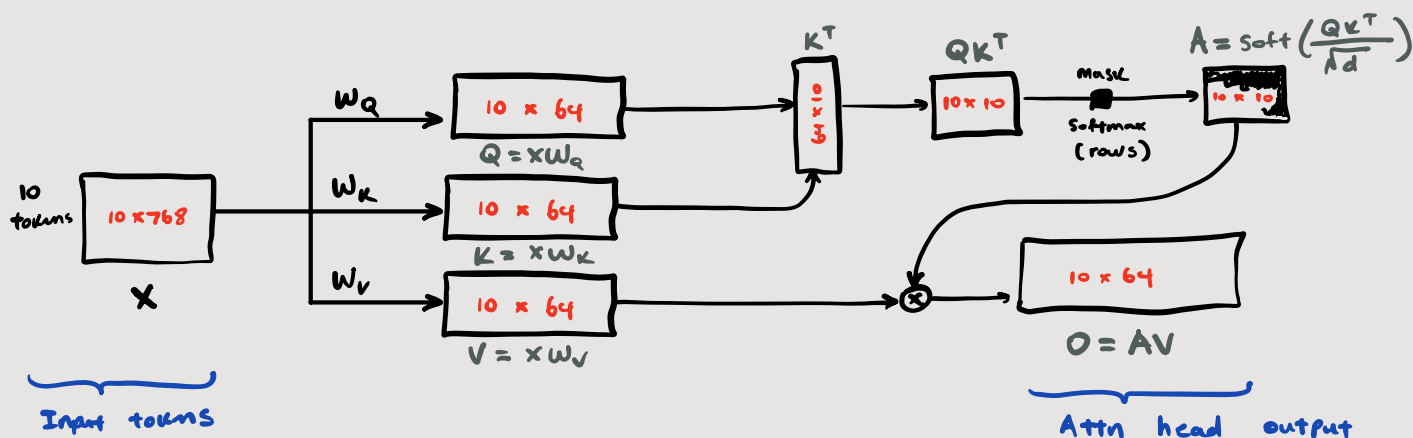
$$\left[e^{s_1-M} \vec{v}_1 + e^{s_2-M} \vec{v}_2 + \dots \right]$$

PROOF

$$\begin{aligned}
 & \text{old part} \quad \text{new part} \\
 & \text{Correction}_{\text{old}} \times \left(\text{denom}_{\text{tile}} \times \text{output}_{\text{tile}} \right) + \text{Correction}_{\text{new}} \times \text{unnorm. exps} \times \vec{V}_{\text{tile}} \\
 & = (e^{m_1-M}) \cdot \left(\cancel{\frac{1}{d}} \times \frac{e^{s_1-m_1} \vec{v}_1 + e^{s_2-m_1} \vec{v}_2}{\cancel{d}} \right) + (e^{m_2-M}) \times \left([e^{s_3-m_2}, e^{s_4-m_2}] \cdot \begin{bmatrix} \vec{v}_3 \\ \vec{v}_4 \end{bmatrix} \right) \\
 & = (e^{m_1-M}) (e^{s_1-m_1} \vec{v}_1 + e^{s_2-m_1} \vec{v}_2) + (e^{m_2-M}) (e^{s_3-m_2} \vec{v}_3 + e^{s_4-m_2} \vec{v}_4) \\
 & = \boxed{\sum_i e^{s_i-M} \vec{v}_i}
 \end{aligned}$$

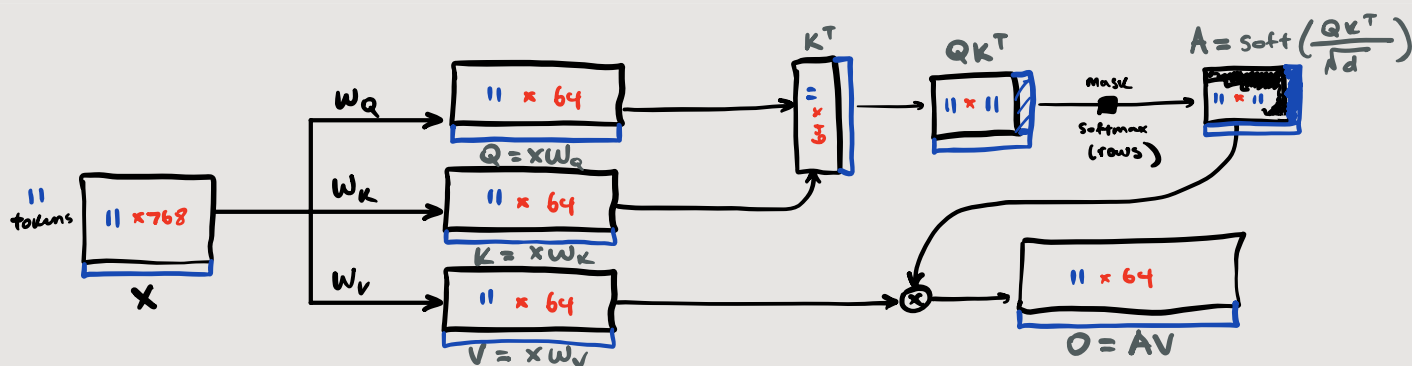
KV - CACHING

This diagram covers 1 head of attention!



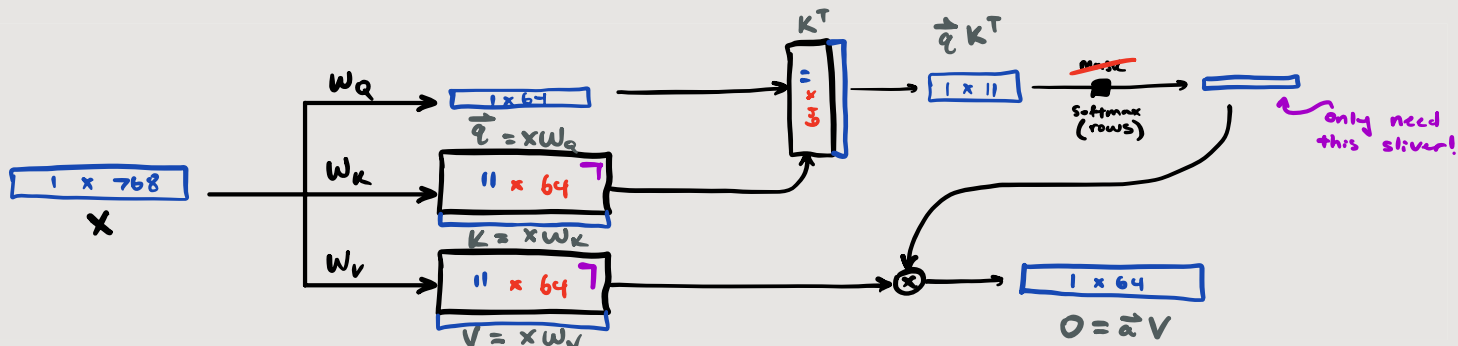
- Let's dub this specific computation "prefill"

- During inference, we add 1 more token!



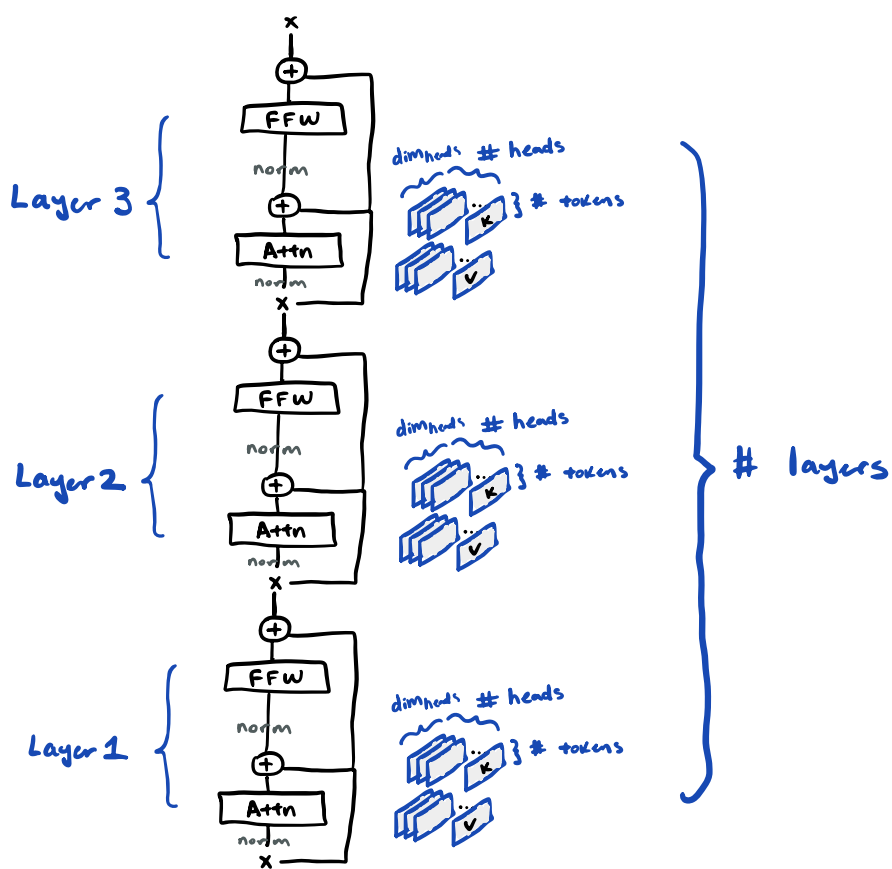
- Notice: black parts are unchanged. You're just adding the blue slices!

↳ Let's remove anything unnecessary to get the new O



NOTICE: we really only 'needed' 11×64 & 11×64 . Let's store these & call them our KV-cache!

- BTW, let's dub this single token/vector computation "decode"



Size of KV cache

$$\text{Model Layers} \times \left(\# \text{ toks} \times \# \text{ KV heads} \times \text{dim heads} \right) \times \text{bits per number}$$

Prefill - Decode

PREFILL

GEMM

$S \times d \times d \times S$

$S \times \text{full} \rightarrow S \times \text{full}$

given $\vec{t}_1, \vec{t}_2 \dots$ we do:

$$\text{softmax}(QK^T) \cdot V$$

DECODE

$$K_{\text{Cache}} = \begin{bmatrix} \vec{k}_1 & \dots & \vec{k}_n & | & \vec{k}_{n+1} \end{bmatrix}$$

$d \times \text{seq}$

$$\begin{bmatrix} \text{prev } q_s \end{bmatrix}$$

$$\begin{bmatrix} \text{Prior Scores} \end{bmatrix}$$

Prior Scores

no need to recompute!

$$\text{new } \vec{q} = \begin{bmatrix} -\vec{q}_{n+1}- \end{bmatrix}$$

$$\begin{bmatrix} -\text{new scores}- \end{bmatrix}$$

$1 \times \text{seq}$

$1 \times d$

$\downarrow \text{softmax}$

$$\begin{bmatrix} -\vec{v}_1- \\ \vdots \\ -\vec{v}_{n+1}- \end{bmatrix}$$

$\text{seq} \times \text{full}$

$$\begin{bmatrix} -\text{new scores}- \end{bmatrix} \rightarrow$$

$$\begin{bmatrix} \text{new output vector} \end{bmatrix}$$

$1 \times \text{full}$

$1 \times \text{seq}$

v